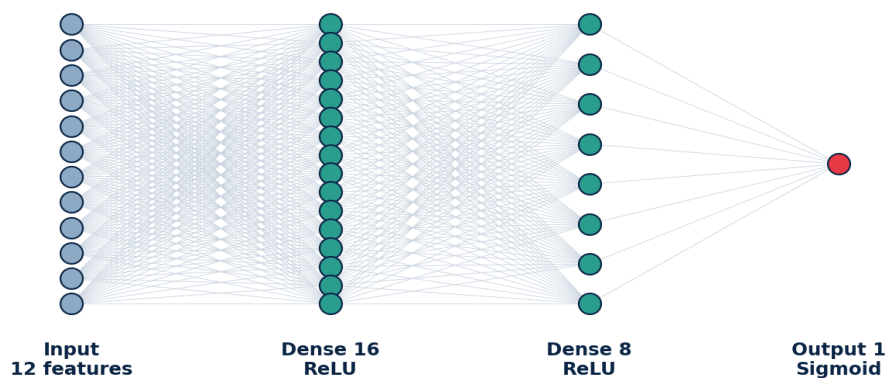


Phishing Website Detection using Artificial Neural Network (ANN)

A Mini Project Report

Data Structures Laboratory

ANN Architecture: 12 - 16 - 8 - 1 (353 trainable parameters)



Submitted by	Name / Roll No.: _____
Class / Division	_____
Guide	_____
Academic Year	_____

Contents

- 1. Abstract
- 2. Introduction
- 3. Literature Survey
- 4. Methodology
- 5. Dataset
- 6. ANN Architecture
- 7. Implementation and Data Structures
- 8. User Interface
- 9. Results
- 10. Conclusion and Future Scope
- 11. References

1. Abstract

Phishing is a cyber attack in which a fake website imitates a trusted one in order to steal sensitive information such as passwords, bank details and personal data. Traditional defences such as blacklists can only block sites that have already been reported, so newly created phishing pages often remain undetected for hours or days. This mini project builds a lightweight machine learning based detector that decides whether a website is **Phishing** or **Legitimate** using only 12 engineered URL and domain features.

The system uses the Kaggle *Phishing Dataset for Machine Learning* (file `Phishing_Legitimate_full.csv`, 10,000 websites: 5,000 phishing and 5,000 legitimate, described by 48 numeric features). Twelve URL-based features are selected, cleaned, split 80:20 and standardised. A small Artificial Neural Network with architecture 12 - 16 - 8 - 1 (ReLU hidden layers and a sigmoid output) is trained with the Adam optimiser and binary cross-entropy loss for 25 epochs with batch size 32. A reusable function `predict_website()` returns the predicted class, the confidence and a risk level (Low, Medium, High), and a four-page Streamlit application presents a prediction report with a risk meter, a suspicious-feature checklist and a recommendation.

The project also demonstrates core Data Structures ideas: arrays for feature vectors, strings for URL analysis, hash maps (dictionaries) for feature lookup and CSV as tabular storage. The project is deliberately kept simple: no web scraping, no database and no authentication.

2. Introduction

The web is used daily for banking, shopping and education, and attackers exploit this trust. In a phishing attack the victim receives a link (by e-mail, SMS or social media) that leads to a page that looks like a genuine login or payment page. Anything typed there goes straight to the attacker. Because creating a copy of a website is cheap, thousands of new phishing sites appear every day.

2.1 Problem Statement

Given the values of a set of URL and domain characteristics of a website, automatically classify the site as phishing or legitimate, report how confident the model is, and translate that confidence into an easy-to-understand risk level for the user.

2.2 Objectives

- Load and clean the phishing dataset and select 12 informative features.
- Perform exploratory data analysis (class balance, correlation, feature distributions).
- Design, train and evaluate a simple ANN classifier.
- Provide a reusable prediction function returning prediction, confidence and risk level.
- Build a Streamlit interface that explains the result to a non-technical user.

2.3 Scope

The project works on pre-computed features rather than raw web pages. It does not crawl websites, does not include a browser extension and does not use a database. This keeps the code small and easy to explain in a laboratory viva.

3. Literature Survey

Blacklist and whitelist methods. Browsers keep lists of known malicious addresses. They are accurate for known sites but cannot recognise a site that has just been created, and the lists must be updated continuously.

Heuristic rule-based methods. Rules such as 'URL contains @', 'URL uses an IP address' or 'domain is very young' were proposed as indicators of phishing. Rules are easy to understand but are chosen by hand and cannot capture combinations of weak signals.

Machine learning methods. Mohammad, Thabtah and McCluskey (2014) studied neural-network based prediction of phishing sites using a widely used feature set (UCI Machine Learning Repository). Tan (2018) published the dataset used in this project, in which features extracted from URLs and page content were evaluated for phishing detection. Sahingoz et al. (2019) applied several machine learning algorithms to URL-based features. Common classifiers in the literature include Decision Trees, Random Forests, Support Vector Machines and Neural Networks.

Observation. Feature-based learning generalises to unseen sites better than lists, and a small ANN is enough for a tabular problem with a dozen inputs, which is why it was chosen here.

4. Methodology

The project follows a standard supervised-learning pipeline. Each stage is implemented in a separate Python file so that it can be explained and tested independently.

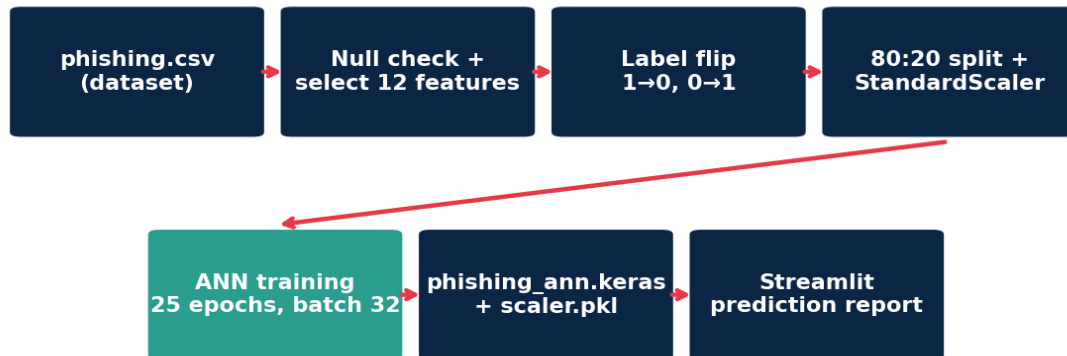


Figure 1: End-to-end pipeline

4.1 Data Preprocessing (src/preprocess.py)

- **Load** the CSV file into a Pandas DataFrame.
- **Null check** using `df.isnull().sum()`; rows with nulls would be dropped.
- **Feature selection**: keep only the 12 chosen columns plus CLASS_LABEL.
- **Label mapping**: in the CSV CLASS_LABEL 1 = phishing and 0 = legitimate; the project flips it so that 0 = Phishing and 1 = Legitimate.
- **Split**: 80% training and 20% testing (stratified so both parts keep the same class ratio).
- **Scaling**: StandardScaler is fitted on the training part only, then applied to both; it is saved as `scaler.pkl` so that the prediction module uses exactly the same scaling.

4.2 Exploratory Data Analysis

Four visual summaries are produced automatically and saved in the `assets` folder: a class-distribution pie chart, a correlation heatmap, per-feature distribution plots (split by class) and, after training, a confusion matrix. The EDA notebook (`notebook/EDA.ipynb`) walks through the same steps interactively.

4.3 Training and Evaluation (src/train.py)

The ANN is trained for 25 epochs with a batch size of 32; 10% of the training data is held out for validation so that over-fitting can be seen in the training curves. The untouched 20% test set is used only once, for the final metrics: accuracy, precision, recall, F1-score and the confusion matrix. The model is saved as `phishing_ann.keras`.

4.4 Prediction Logic (src/predict.py)

The sigmoid output p is interpreted as the probability that the site is legitimate. The predicted class is Legitimate when p is at least 0.5. Confidence is $\max(p, 1 - p)$ as a percentage and the **risk score** is $(1 - p) \times 100$, i.e. the probability of phishing.

Risk score	Risk level	Meaning
0 - 30	Low	Very likely legitimate
31 - 70	Medium	Uncertain - verify manually
71 - 100	High	Very likely phishing

5. Dataset

The dataset is the Kaggle *Phishing Dataset for Machine Learning* (Phishing_Legitimate_full.csv, renamed phishing.csv). It contains 10,000 website records: 5,000 phishing and 5,000 legitimate, so the classes are perfectly balanced. Each record has an id, 48 numeric features extracted from the URL and the page, and the label CLASS_LABEL. Features are counts (for example the number of dots), 0/1 flags (for example whether '@' is present) or fractions between 0 and 1.

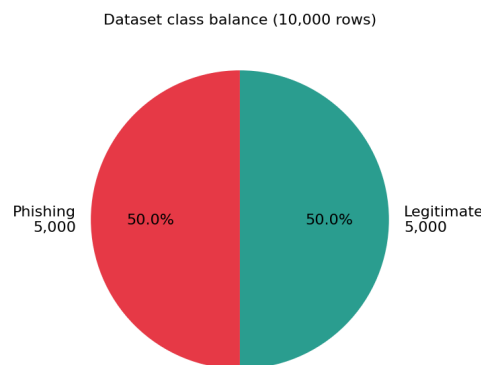


Figure 2: Class balance of the dataset (5,000 / 5,000)

5.1 Features Used

Twelve features that are easy to explain and cover URL structure, domain information and page behaviour were selected:

Feature	Meaning	Values
NumDots	Number of '.' in the URL	count
SubdomainLevel	Number of sub-domain levels	count
PathLevel	Number of '/' levels in the path	count
UrlLength	Total characters in the URL	count
NumDash	Number of '-' in the URL	count
AtSymbol	'@' present in the URL	0 / 1
NumNumericChars	Number of digits in the URL	count
NoHttps	HTTPS not used	0 / 1
IpAddress	IP address used as the host	0 / 1
NumSensitiveWords	Words such as login, secure, bank	count
HostnameLength	Characters in the host name	count
PctExtHyperlinks	Fraction of links to other domains	0 - 1

5.2 Target Variable

The column CLASS_LABEL is 1 for phishing and 0 for legitimate in the original file. The project flips it (0 = Phishing, 1 = Legitimate) so that the single sigmoid output neuron gives the probability of a legitimate site. After an 80:20 stratified split, the training set holds 8,000 records and the test set 2,000 records.

6. ANN Architecture

An Artificial Neural Network is a set of connected layers of simple units (neurons). Each neuron computes a weighted sum of its inputs, adds a bias and passes the result through an activation function. During training the weights are adjusted by gradient descent so that the loss decreases.

ANN Architecture: 12 - 16 - 8 - 1 (353 trainable parameters)

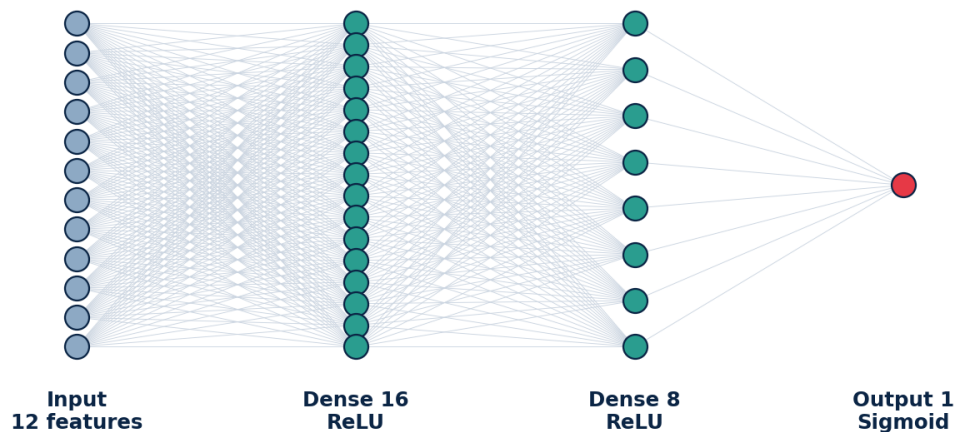


Figure 3: 12 - 16 - 8 - 1 network

Layer	Units	Activation	Parameters
Input	12	-	0
Dense 1	16	ReLU	$12 \times 16 + 16 = 208$
Dense 2	8	ReLU	$16 \times 8 + 8 = 136$
Output	1	Sigmoid	$8 \times 1 + 1 = 9$
Total			353

6.1 Design Choices

- **ReLU** in hidden layers: fast to compute and avoids vanishing gradients.
- **Sigmoid** at the output: gives a value between 0 and 1 that is read as a probability.
- **Binary cross-entropy**: the natural loss for two-class problems.
- **Adam**: adaptive optimiser that works well without much tuning.
- **Small network**: 353 parameters are enough for 12 inputs and train in seconds on a laptop.

6.2 Training Configuration

Parameter	Value
Optimizer	Adam
Loss	Binary cross-entropy
Metric	Accuracy
Epochs	25
Batch size	32
Train / test split	80 : 20 (stratified, random_state = 42)
Validation	10% of training data
Scaling	StandardScaler (fit on training data)

7. Implementation and Data Structures

The project is written in Python 3.11 using Pandas, NumPy, Scikit-learn, TensorFlow/Keras, Streamlit, Matplotlib and Seaborn.

File	Purpose
src/preprocess.py	Load CSV, null check, feature selection, label mapping, split, scaling, EDA plots
src/train.py	Build and train the ANN, compute metrics, save model, training curves and confusion matrix
src/predict.py	predict_website(features_dict), risk levels, suspicious-feature checklist, recommendations
app.py	Streamlit interface with four pages
notebook/EDA.ipynb	Interactive exploratory analysis

7.1 Data Structures Relevance

Data structure	Use in this project
Array / list	The ordered list of 12 feature names and the 12-value feature vector fed to the ANN; NumPy arrays hold the scaled training data. Access by index is $O(1)$.
String	URL analysis: length, number of dots, digits and '-', presence of '@', IP-address host and sub-domain levels are all string operations that produce the feature values.
Hash map (dictionary)	The input of predict_website() is a dictionary {feature name: value}; FEATURE_INFO maps each feature to its description and value labels. Average lookup time is $O(1)$.
CSV (tabular storage)	The dataset is stored as rows and columns in phishing.csv and loaded into a DataFrame.

7.2 Prediction Function

predict_website(features_dict) validates that all 12 keys are present, converts the dictionary into a one-row feature vector in the fixed order of the FEATURES list, applies the saved

scaler, runs the model and returns a dictionary with the keys *prediction*, *confidence*, *risk_score* and *risk_level*. Because the function is independent of the interface, it can be reused from a script, a notebook or the Streamlit app.

8. User Interface

The Streamlit application has four pages selectable from the sidebar:

Page	Content
Home	Project overview, model summary, usage steps and data-structure notes.
URL Feature Input	One numeric input per feature (with a hint for each), plus buttons that load a legitimate-like or phishing-like example.
Prediction Report	Legitimate/Phishing verdict, confidence percentage, colour risk meter, suspicious-feature checklist (tick / cross per feature) and a recommendation.
Analytics	Accuracy, precision, recall, F1-score and the saved charts: class distribution, correlation heatmap, feature distributions, training curves and confusion matrix.

8.1 Recommendation Rules

- **Low risk:** the site looks safe; continue to check the address bar.
- **Medium risk:** verify the URL manually and avoid entering passwords or card details.
- **High risk:** do not enter personal data, leave the site and report it.

8.2 Running the Project

Install the packages with `pip install -r requirements.txt`, place the Kaggle file at `dataset/phishing.csv`, run `python src/train.py` once and start the interface with `streamlit run app.py`.

9. Results

Running `python src/train.py` on the real dataset produces all result artefacts automatically. The numbers are written to `assets/metrics.json` and shown on the Analytics page. **Copy the values into Table 3 after training**; they are not filled in here because they depend on the dataset file used and on the training run.

Metric	Value on test set (20%)
Accuracy	_____ %
Precision	_____ %
Recall	_____ %
F1-score	_____ %

Table 3: Test-set performance (fill in from `assets/metrics.json`)

9.1 Generated Figures

The following images are created by the code and can be pasted here for the final submission:

- **`assets/class_distribution.png`** - pie chart of phishing versus legitimate records.
- **`assets/correlation_heatmap.png`** - correlation between the 12 features and the target.
- **`assets/feature_distributions.png`** - value counts of each feature split by class.
- **`assets/training_curves.png`** - training and validation accuracy and loss over 25 epochs.
- **`assets/confusion_matrix.png`** - true/false positives and negatives on the test set.

9.2 How to Interpret the Results

The confusion matrix has four cells. A *false negative* (a phishing site predicted as legitimate) is the most dangerous error, so recall on the Phishing class should be checked as carefully as overall accuracy. If validation accuracy stops improving while training accuracy keeps rising, the model is over-fitting; with a network this small and 25 epochs that is not expected. Features such as `PctExtHyperlinks`, `NoHttps`, `NumDash` and `UrlLength` are expected to carry much of the predictive signal, which can be confirmed in the heatmap.

10. Conclusion and Future Scope

This project shows that a small ANN with only 12 easily understood features can be used to flag phishing websites, and that a machine learning model can be wrapped into a simple, explainable application. The complete flow from raw CSV to a user-friendly prediction report was implemented with clear, short modules: preprocessing, training, prediction and interface. The project also connects machine learning with basic data structures - arrays, strings, hash maps and tabular files - which are the building blocks of every step.

10.1 Limitations

- Features must be supplied by the user; they are not extracted from a live URL.
- The dataset is a snapshot; phishing techniques change over time.
- Only 12 of the 48 available features are used.

10.2 Future Scope

- Automatic feature extraction from a raw URL string using string processing and regular expressions.
- Using all 48 features and comparing with Random Forest, SVM and gradient boosting.
- Browser extension or REST API for real-time protection.
- Model explainability (SHAP/LIME) and periodic retraining with new phishing data.

11. References

- R. Mohammad, F. Thabtah, L. McCluskey, 'Predicting phishing websites based on self-structuring neural network', *Neural Computing and Applications*, 2014.
- C. L. Tan, 'Phishing Dataset for Machine Learning: Feature Evaluation', Mendeley Data, 2018 (Kaggle: Phishing Dataset for Machine Learning).
- UCI Machine Learning Repository - Phishing Websites Data Set (Mohammad and McCluskey).
- O. K. Sahingoz et al., 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 2019.
- TensorFlow/Keras, Scikit-learn and Streamlit official documentation.